Data Structures Assignment: Linear & Hierarchical Structures

This assignment covers three fundamental pillars of computer science: **Singly Linked Lists**, **Binary Search Trees (BST)**, and **Big O Notation**.

Question 1: Singly Linked Lists

Task: Define a Singly Linked List, explain its structure, provide a Python implementation for node insertion, and calculate the time complexity for searching an element.

Definition & Explanation

A **Singly Linked List** is a linear data structure where elements are not stored at contiguous memory locations. Instead, each element (called a **Node**) contains two parts:

1. **Data:** The actual value stored.
2. **Next:** A pointer/reference to the next node in the sequence.

Unlike arrays, linked lists allow for efficient insertion and deletion because you don't need to shift elements; you simply update the pointers.

Mathematics: Time Complexity

To find an element in a linked list of size n, in the worst case, we must visit every node starting from the head.

$$₩text{Time Complexity} = O(n)$$

Code Implementation (Python)

```
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None


class LinkedList:
    def __init__(self):
        self.head = None
```

```
    def insert_at_end(self, data):
        new_node = Node(data)
        if not self.head:
            self.head = new_node
            return
        temp = self.head
        while temp.next:
            temp = temp.next
        temp.next = new_node
```

Question 2: Binary Search Trees (BST)

Task: Explain the properties of a BST, provide the mathematical logic for its height, and show a recursive search function.

Definition & Explanation

A **Binary Search Tree** is a rooted binary tree data structure with the following properties:

- The **left subtree** of a node contains only nodes with keys lesser than the node's key.
- The **right subtree** of a node contains only nodes with keys greater than the node's key.
- The left and right subtree each must also be a binary search tree.

Mathematics: Tree Height

In a perfectly balanced BST with n nodes, the height h is logarithmic:

$$h = ₩log_{2}(n)$$

This efficiency is why BSTs are preferred for fast lookups, as each step eliminates half of the remaining tree.

Code Implementation (Python)

```
def search(root, key):
    # Base Cases: root is null or key is present at root
    if root is None or root.val == key:
        return root

    # Key is greater than root's key
    if root.val < key:
        return search(root.right, key)

    # Key is smaller than root's key
    return search(root.left, key)
```

Question 3: Stack Operations & Postfix Evaluation

Task: Define the "LIFO" principle, explain the mathematical evaluation of a postfix expression using a stack, and provide the code for the push and pop operations.

Definition & Explanation

A **Stack** is an abstract data type that serves as a collection of elements, with two main principal operations: **Push** (adds an element) and **Pop** (removes the most recently added element). It follows the **Last-In, First-Out (LIFO)** principle.

Mathematics: Postfix Evaluation

Stacks are used to evaluate postfix expressions (e.g., $3 ₩ 4 ₩ +$).

Given an expression $A ₩ B ₩ ₩text{op}$:

1. Push A
2. Push B
3. When operator $₩text{op}$ is encountered: $₩text{Result} = ₩text{pop}(B) ₩ ₩text{op} ₩ ₩text{pop}(A)$

For example, $3 ₩ 4 ₩ +$ becomes:

$$3 + 4 = 7$$

```
class Stack:
    def __init__(self):
        self.stack = []

    def push(self, item):
        self.stack.append(item)

    def pop(self):
        if not self.is_empty():
            return self.stack.pop()
        return "Stack Underflow"

    def is_empty(self):
        return len(self.stack) == 0
```